

BPC_03_VRUIWidget Component

1. Component Overview

Purpose

BPC_03_VRUIWidget is a VR UI interaction component that manages the laser rays on both hands, enables/disables the Widget Interaction components, and simulates trigger clicks. With this component, players can point the controller ray at buttons on 3D Widgets and click them.

Design

- Symmetric left/right hand design; an enum controls the hand mode (Both / Only Left / Only Right)
- The Widget Interaction component interacts on the 3D Widget channel; 3D Widgets must enable the collision response on that channel
- Uses the Enhanced Input system with a dedicated IMC_Widget mapping at priority 2

2. Public Variables

The following variables are exposed for customizing the UI interaction:

Variable	Type	Description
Interaction Distance	Float	Ray interaction distance, controls the ray length and the Widget Interaction detection range
Widget Mode	E_VRWidgetMode enum	Hand mode: Both / Only

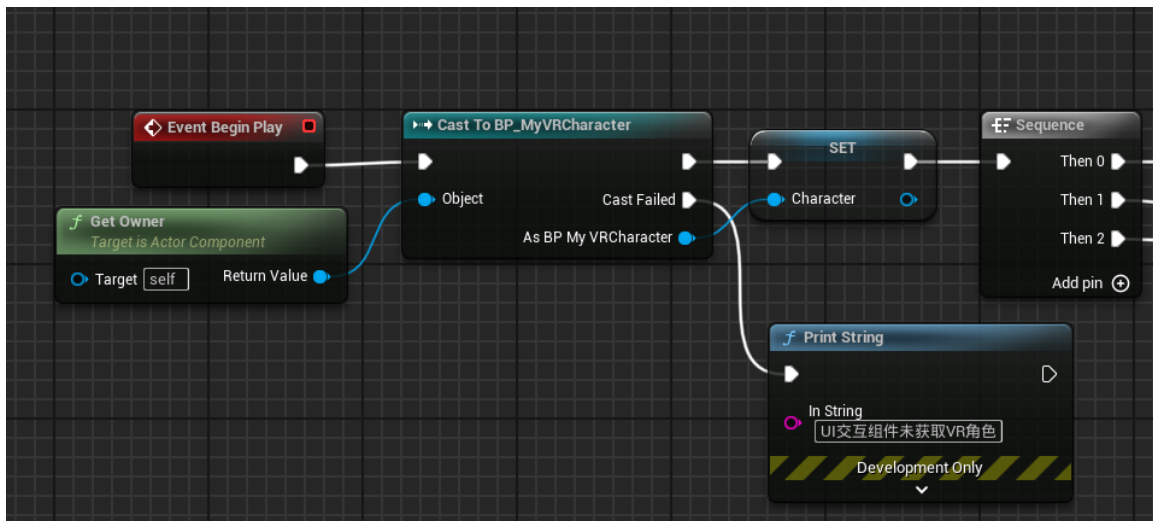
		Left Hand / Only Right Hand
Can Open UIWidget?	Boolean	Whether opening the UI is allowed, for external condition control

3. Initialization (BeginPlay)

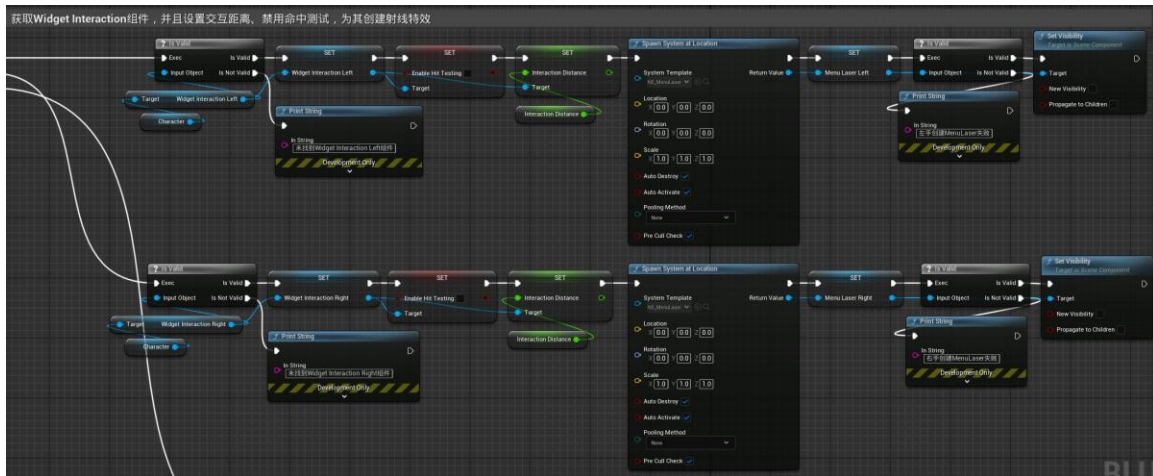
Description

During initialization the component does three things: obtain the VR character reference and cache the Widget Interaction components, create the Niagara ray effects, and bind the UI interaction input mapping.

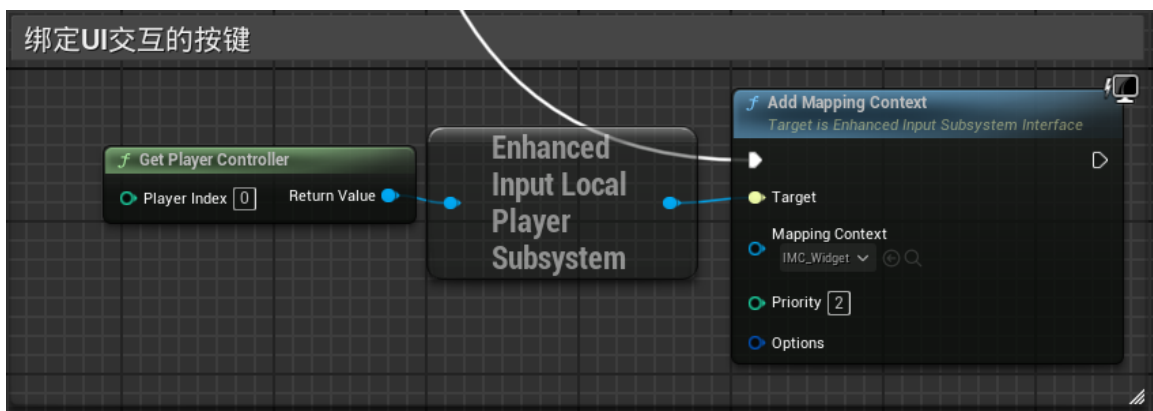
Blueprint screenshot - getting the character reference



Blueprint screenshot - getting the interaction components and creating the rays



Blueprint screenshot - binding the input mapping



Notes

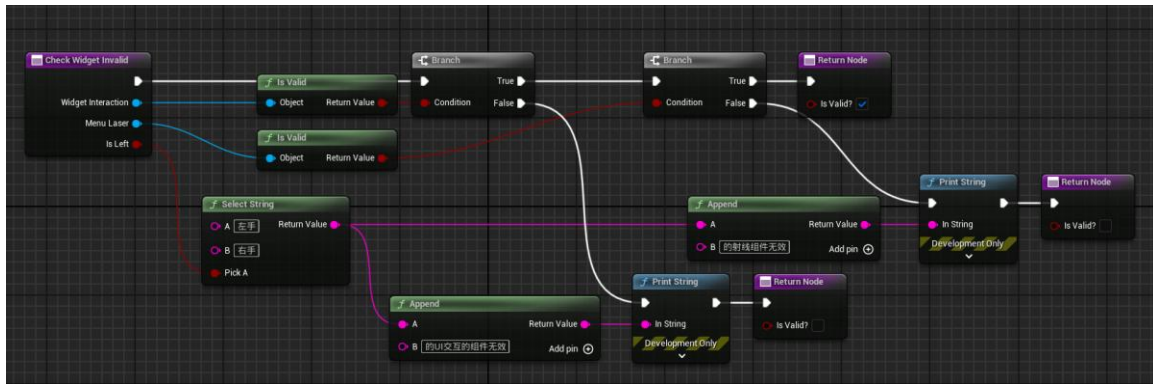
- Enable Hit Testing is initially false and only turned on in OpenHand
- IMC_Widget priority is 2 so UI input takes precedence over gameplay input
- The ray effects start hidden and are only shown in OpenHand

4. Check Widget Invalid Function

Description

Validates the Widget Interaction and Menu Laser components. Each is checked independently; whichever is invalid prints its own failure message, and a boolean is returned.

Blueprint screenshots



Key points

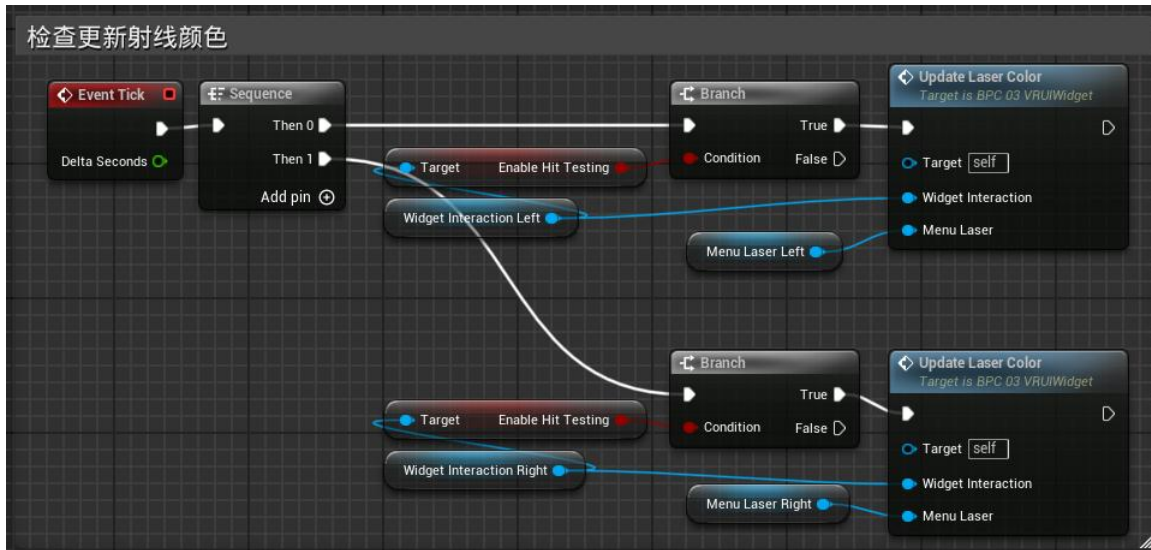
- The two components are checked independently, each printing its own error message
- The Is Left parameter distinguishes left/right hand in the error messages
- Every Open/Close operation calls this function first for validity checking

5. Tick - Updating the Ray Color

Description

Every frame it checks Enable Hit Testing on both hands and, only when enabled, calls Update Laser Color, which sets the ray color (green/white) based on whether it points at an interactable Widget.

Blueprint screenshots



Update Laser Color function logic

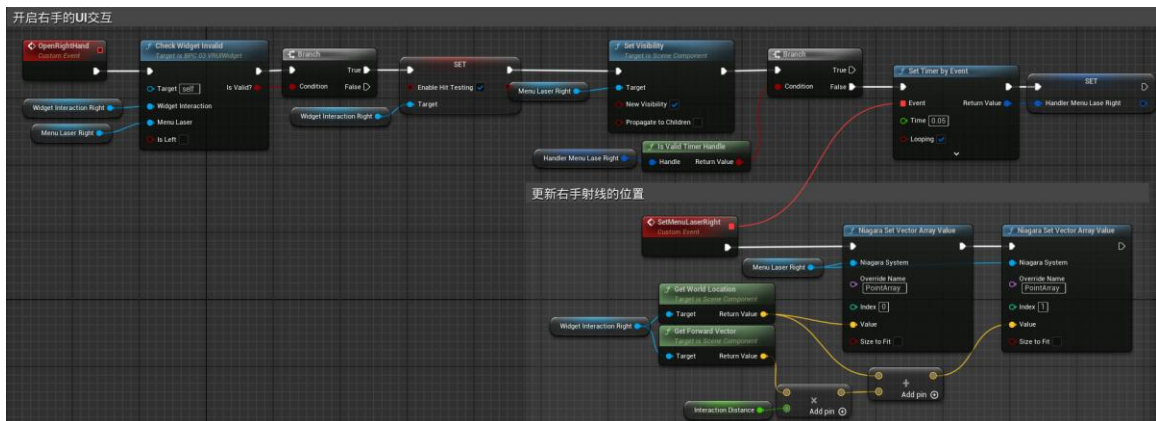
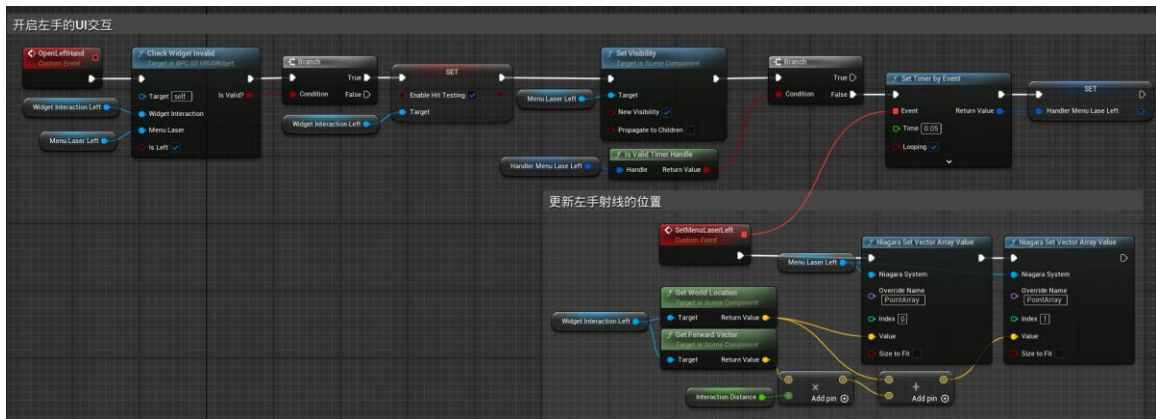
- Is Over Interactable Widget -> True: Set Niagara Variable (LaserColor) = green (0,1,0,1)
- Is Over Interactable Widget -> False: Set Niagara Variable (LaserColor) = white (1,1,1,1)
- Is Over Interactable Widget distinguishes interactable components (buttons) from non-interactive ones (text, images)

6. Open Hand / Close Hand

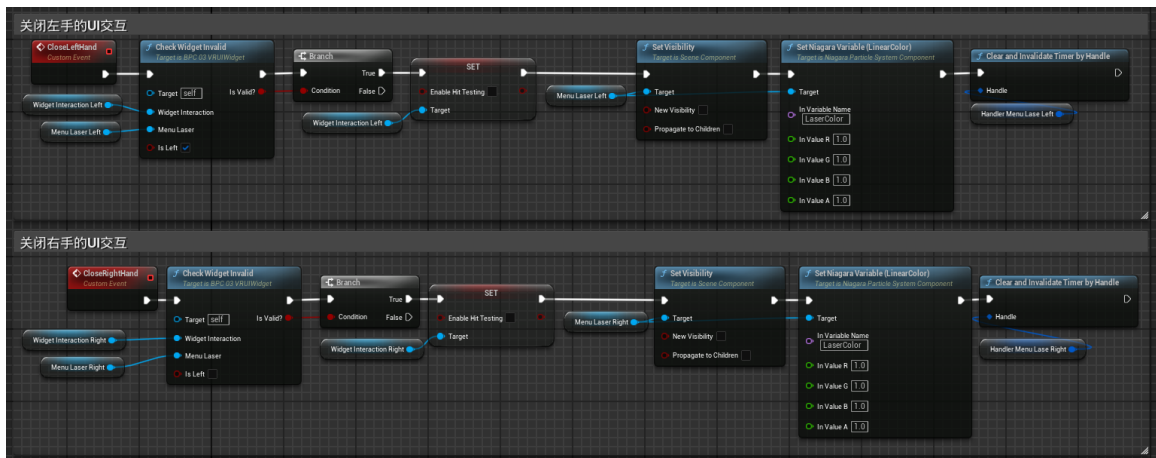
Description

OpenHand enables one hand's UI interaction: turn on hit testing, show the ray, and start a Timer that keeps updating the ray positions. CloseHand disables it: turn off hit testing, hide the ray, reset the color, and clear the Timer.

Blueprint screenshot - Open Hand



Blueprint screenshot - Close Hand



Key points

- OpenHand uses Is Valid Timer Handle to prevent creating duplicate Timers
- Ray positions are set through Niagara's PointArray: [0] is the start (controller position), [1] is the end (a distance ahead)

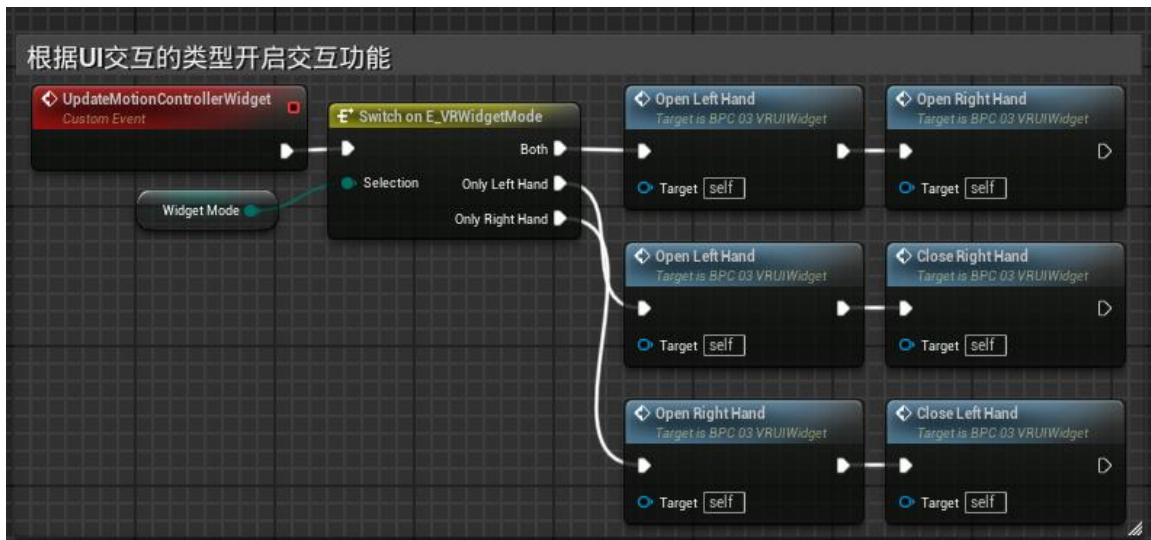
- CloseHand fully cleans up: disable testing + hide the ray + reset the color + clear the Timer
- The Timer runs every 0.05 seconds (20Hz), balancing smoothness and performance

7. UpdateMotionControllerWidget - Mode Switching

Description

Switches both hands' UI interaction state according to the Widget Mode enum, calling OpenHand / CloseHand combinations to implement the three modes.

Blueprint screenshots

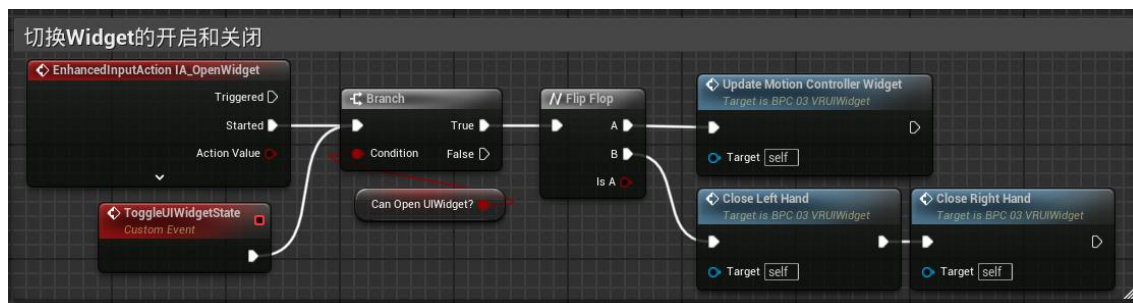
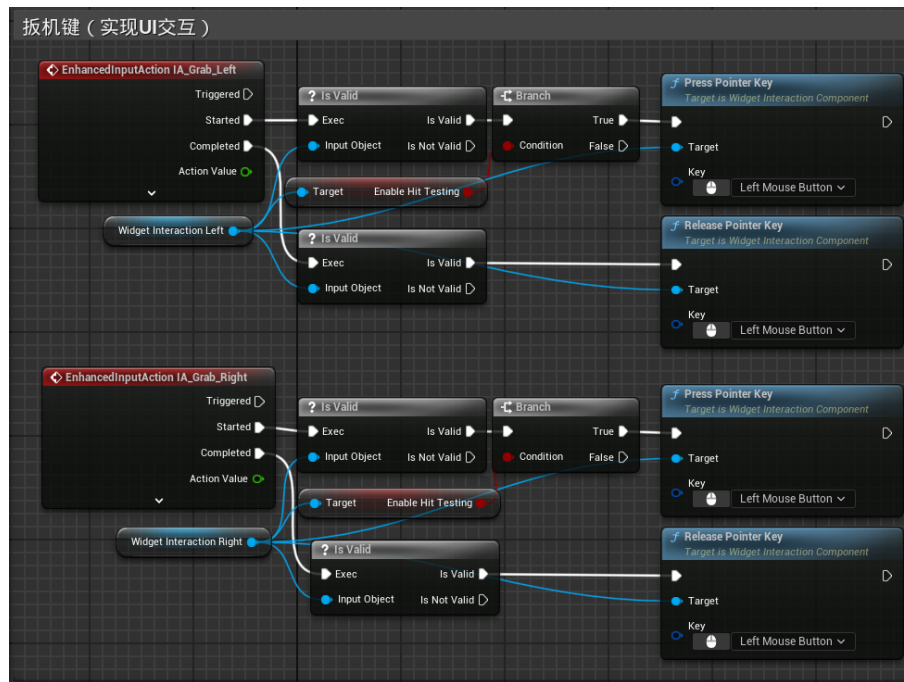


8. Input Handling

Description

Handles two kinds of input: the trigger simulating mouse clicks (Press/Release Pointer Key), and the UI toggle (IA_OpenWidget + Flip Flop).

Blueprint screenshots



Trigger click (IA_Grab_Left / IA_Grab_Right)

- Maps the VR trigger to the left mouse button so standard UMG buttons work in VR without modification

UI toggle (IA_OpenWidget)

- A Flip Flop implements the toggle; each press switches between on and off
- The Can Open UIWidget variable allows external condition control, e.g. blocking the menu during gameplay

9. 3D Widget Collision Preset

Description

The 3D Widget's (Widget Component's) collision preset must be set correctly; otherwise the Widget Interaction component on the VR character cannot detect the 3D Widget and ray interaction stops working.

Background

- The VR character's Widget Interaction component performs its ray trace on the "3D Widget" collision channel
- In the 3D Widget's collision preset, that channel must be set to Overlap or Block to be detectable
- If the 3D Widget's preset is Ignore for that channel, Widget Interaction cannot recognize the UI

▼ Collision			
Simulation Generates Hit Events	☐		
Phys Material Override	None	None ▼	
Generate Overlap Events	☒		
Can Character Step Up On	Yes ▼		
▼ Collision Presets	Custom... ▼		
Collision Enabled	Query Only (No Physics Collision) ▼		
Object Type	WorldDynamic ▼		
	Ignore	Overlap	Block
Collision Responses ?	☒	☒	☒
Trace Responses			
Visibility	☒	☐	☐
Camera	☒	☐	☐
3DWidget	☐	☐	☒
Object Responses			
WorldStatic	☐	☒	☐
WorldDynamic	☐	☒	☐
Pawn	☐	☒	☐
PhysicsBody	☐	☒	☐
Vehicle	☐	☒	☐
Destructible	☐	☒	☐

Notes

- Object Type can stay at its default (WorldDynamic or custom); what matters is the response on the 3D Widget channel
- If 3D Widgets are created through a Blueprint, the preset can be set in the Blueprint editor, avoiding manual setup on every instance
- Multiple 3D Widgets should share the same collision preset for easier management